Diving into Deep Learning

An Exploration with the Neural Network Playground: Understanding Neural Network Architecture and Training Dynamics

by Ziqi Zhao

Assignment Overview: Neural Network Components

What We're Doing

To understand and define the core components of Neural Networks by examining their structure and functionality.

Component Definition Goals

Analyzing how changes in data (type, noise, features) and architecture (layers, neurons) impact the model's training time, complexity, and performance.

Visual Structure Analysis

I tested various settings on the 'Circle' and 'Spiral' data sets to see the contrast between simple and complex problems.

Learning Through Visualization

Understanding the relationship between problem complexity, network architecture, and training performance in neural networks.

The Building Blocks: Layers, Neurons, & Weights

Understanding the core components that make up neural networks and their fundamental roles in processing information and learning patterns from data.

Component	Definition	Role/Function
Layers	Hierarchical organization of neurons that process information in distinct stages, typically consisting of input layers (receiving raw data), hidden layers (performing transformations), and output layers (producing final predictions)	Facilitate the flow of information through the network, enabling progressive feature extraction and abstraction at each level while maintaining separation of concerns between different computational tasks
Neurons (Nodes)	Nonlinear computational units that apply activation functions (e.g., ReLU, sigmoid) to weighted sums of inputs, mimicking biological neurons' all-or-nothing firing behavior through threshold-based activation	Perform the fundamental mathematical operations that enable networks to learn complex patterns by introducing nonlinearity, allowing the modeling of intricate relationships between input features and output predictions
Weights (w)	Trainable parameters that quantify the strength and direction of connections between neurons, initialized randomly and optimized through backpropagation algorithms like gradient descent	Encode the network's learned knowledge by determining how input signals propagate through the architecture, with weight adjustments during training minimizing loss functions to improve prediction accuracy

The Engine Room: Functions for Learning

Mathematical functions and algorithms that enable neural networks to learn complex patterns and make accurate predictions through iterative optimization.

Component	Definition	Role/Function
Activation Functions	Non-linear mathematical transformations applied to the weighted sum of inputs at each neuron (e.g., ReLU: max(0,x), Sigmoid: 1/(1+e^-x), Tanh: (e^x-e^-x)/(e^x+e^-x))	Introduce non-linearity to enable learning of complex patterns; determine neuron firing rate; prevent linear collapse in deep networks; different functions suit different tasks (ReLU for hidden layers, Sigmoid for binary classification)
Loss Functions	Objective functions that quantify the discrepancy between model predictions and ground truth values (e.g., MSE: Σ(y_pred-y_true)²/n, Cross-Entropy: -Σy_true*log(y_pred))	Provide differentiable performance metrics for optimization; guide weight updates during backpropagation; choice affects model behavior (MSE for regression, Cross-Entropy for classification); may include regularization terms
Optimization Algorithms	Iterative methods that minimize loss functions by adjusting network parameters (e.g., SGD: θ=θ-η*∇θJ(θ), Adam: adaptive momentum estimation)	Efficiently navigate high-dimensional parameter spaces; balance convergence speed and stability; adaptive methods (Adam, RMSprop) automatically tune learning rates; handle noise and sparse gradients; critical for training deep networks

How Data Flows: Architecture and Processing

Input Layer

Feature Reception: $X = [x_1, x_2, \ldots, x_n]$

Receives raw feature vectors with dimensionality n. No computation occurs; just passes features forward to hidden layers via weighted connections.

1

Hidden Layers

Weighted Sum: $z = \Sigma(w_i x_i) + b$

Activation: $a = \sigma(z)$

Each neuron computes weighted sum of inputs, applies activation function (ReLU, sigmoid, tanh) to introduce non-linearity.

2

Output Layer

Prediction: $\hat{y} = \text{softmax}(z)$

Loss: $L = -\Sigma(y \log(\hat{y}))$

Final layer produces probability distribution. Uses loss function for backpropagation to adjust weights via gradient descent.

3

Component Interactions and Learning Process

Forward Propagation

Data flows through the network layers from input to output

- Input features are processed through hidden layers
- Activation functions introduce non-linearity
- Final output represents network's prediction

Loss Calculation

Comparing network output to target values

- Loss function quantifies prediction error
- Common metrics: MSE, Cross-Entropy
- Provides feedback signal for learning

Backpropagation

Error signals flow backward through network

- Chain rule calculates gradient contributions
- Identifies which weights contributed most to error
- Enables efficient parameter updates

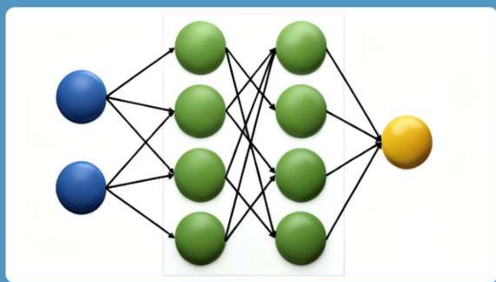
Weight Updates

Optimization adjusts network parameters

- Gradient descent minimizes loss function
- Learning rate controls update magnitude
- Iterative process improves model accuracy

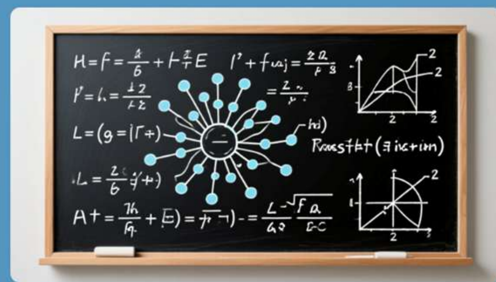
Neural Network Architecture

Network Topology



- Input layer with 784 neurons (28x28 pixels)
- 2 hidden layers with ReLU activation
- Output layer with softmax for classification
- Fully connected architecture

Mathematical Operations



- Z = W·X + b (weighted sum)
- ReLU(Z) = max(0,Z) activation
- Softmax for probability distribution
- Cross-entropy loss function

Learning Dynamics

- Gradient descent optimization
- Backpropagation of errors
- Learning rate scheduling
- Batch normalization effects

Summary: Importance of Neural Network Visualization

Why Visualization Matters

- Makes abstract neural network concepts tangible and concrete
- Reveals hidden patterns in decision boundaries and weight distributions
- Provides immediate feedback on model behavior during training

Component Interaction Insights

- Visualizes how layers transform input data step-by-step
- Shows impact of activation functions on feature representations
- Demonstrates gradient flow and backpropagation dynamics

Theory-Practice Bridge

- Connects mathematical concepts to observable behaviors
- Accelerates debugging by making failures visually apparent
- Enhances intuition for hyperparameter tuning decisions

Educational Value

Visual learning reduces cognitive load and improves knowledge retention

Research Benefits

Enables discovery of novel architectures through visual patterns

Collaboration Tool

Creates common language for explaining complex concepts to diverse audiences